

**UNIVERSIDAD DE SANTIAGO DE CHILE  
DEPARTAMENTO DE INGENIERÍA MECÁNICA  
INGENIERÍA CIVIL MECATRÓNICA**



## **Estructura de Computadores y Sistemas Distribuidos**

### **Evaluación 3 — Caso 2: Línea de Producción con Balanceo Dinámico**

**Sistema distribuido de simulación de una línea conservera con coordinación por colas,  
detección de cuellos de botella en vivo y tablero de operaciones**

#### **Integrantes:**

César Olivares  
Simón García-Huidobro

#### **Profesor:**

Daniel Calderón R.

**Santiago, Chile  
25 de agosto de 2026**



## Índice

|                                                                                |           |
|--------------------------------------------------------------------------------|-----------|
| <b>1. Introducción</b>                                                         | <b>3</b>  |
| <b>2. Arquitectura del sistema</b>                                             | <b>4</b>  |
| 2.1. Servicios . . . . .                                                       | 4         |
| 2.2. Nodos lógicos y su justificación . . . . .                                | 4         |
| 2.3. Flujo de una orden . . . . .                                              | 5         |
| <b>3. Decisiones de diseño</b>                                                 | <b>5</b>  |
| 3.1. Balanceo por demanda: cola <i>pull</i> , no balanceador clásico . . . . . | 5         |
| 3.2. Coordinación y condición de carrera: provocarla antes de resolverla . . . | 6         |
| 3.3. Criterio de cuello de botella: espera, no servicio . . . . .              | 7         |
| 3.4. Persistencia: dos tablas y nada guardado dos veces . . . . .              | 8         |
| 3.5. Interfaz de operación . . . . .                                           | 8         |
| <b>4. Resultados experimentales</b>                                            | <b>8</b>  |
| 4.1. Experimentos 1–3: el cuello se desplaza al escalar . . . . .              | 9         |
| 4.2. Experimento 4: balanceo dinámico con una réplica lenta . . . . .          | 10        |
| 4.3. Experimento 5: saturación y recuperación automática . . . . .             | 11        |
| <b>5. Manejo de errores</b>                                                    | <b>12</b> |
| <b>6. Reproducibilidad y despliegue</b>                                        | <b>12</b> |
| <b>7. Resiliencia: cómo se abordaría</b>                                       | <b>13</b> |
| <b>8. Análisis crítico de dificultades</b>                                     | <b>13</b> |
| <b>9. Supuestos declarados</b>                                                 | <b>14</b> |
| <b>10. Conclusiones</b>                                                        | <b>15</b> |
| <b>A. Contratos completos de las APIs</b>                                      | <b>16</b> |
| A.1. orders-api (puerto 8000) . . . . .                                        | 16        |
| A.2. estacion (una imagen, cuatro despliegues) . . . . .                       | 16        |
| A.3. metrics-api (puerto 8001) . . . . .                                       | 17        |
| A.4. Códigos HTTP en uso . . . . .                                             | 18        |

|                                                                        |           |
|------------------------------------------------------------------------|-----------|
| <b>B. Esquema de datos y momentos de escritura</b>                     | <b>19</b> |
| <b>C. Capturas del tablero y verificación de errores</b>               | <b>21</b> |
| C.1. Verificación de los códigos HTTP y de la caída de Redis . . . . . | 23        |
| <b>D. Salidas crudas de los experimentos</b>                           | <b>25</b> |
| D.1. Experimentos 1–3: el cuello se desplaza al escalar . . . . .      | 25        |
| D.2. Experimento 4: balanceo con una réplica lenta . . . . .           | 26        |
| D.3. Experimento 5: saturación y recuperación automática . . . . .     | 26        |
| <b>E. Historial de commits</b>                                         | <b>28</b> |
| <b>F. Bitácoras por fase</b>                                           | <b>30</b> |

## 1. Introducción

Este informe presenta el diseño, implementación y evaluación experimental de un sistema distribuido que simula una **línea de producción con balanceo dinámico de carga** (Caso 2). El dominio elegido es una planta conservera **ficticia** de jurel en lata de 425 g, inspirada en la industria conservera nacional, con cuatro etapas en serie separadas por **colas de trabajo**: cada cola es, literalmente, la fila de lotes que esperan frente a la etapa siguiente. Si una etapa procesa más lento de lo que le llega trabajo, su fila crece — y esa fila creciente es exactamente lo que este sistema mide y diagnostica en vivo.

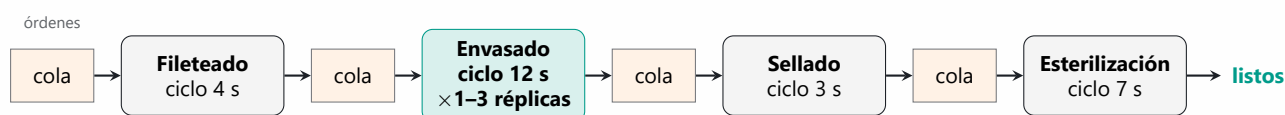


Figura 1: La línea de producción: cuatro etapas en serie unidas por colas de trabajo. Envasado, la etapa más lenta, es la replicada.

Tabla 1: Etapas de la línea y tiempos de ciclo simulados (supuesto declarado, §9).

| Etapa           | Naturaleza            | Ciclo (s/lote) | Réplicas           |
|-----------------|-----------------------|----------------|--------------------|
| Fileteado       | Manual, paralelizable | 4              | 1                  |
| <b>Envasado</b> | Máquina continua      | <b>12</b>      | <b>1–3</b>         |
|                 |                       |                | <b>(replicada)</b> |
| Sellado         | Máquina rápida        | 3              | 1                  |
| Esterilización  | Autoclave, ciclo fijo | 7              | 1                  |

La unidad de trabajo es el **lote** (una cantidad de latas del mismo formato). El sistema cumple los requisitos del enunciado: arquitectura distribuida con más de tres servicios en tres nodos lógicos, una etapa replicada detrás de un mecanismo de balanceo, coordinación del reparto sin condiciones de carrera, detección de cuellos de botella *en vivo*, persistencia, interfaz de operación con interacción real, y despliegue completo con un solo comando (`docker compose up`).

El resultado experimental central (§4) demuestra literalmente la premisa del caso: al escalar envasado de 1 a 2 réplicas, el cuello de botella **se desplaza** de envasado a esterilización y el lead time cae un 21 %; una tercera réplica ya casi no mejora nada, porque el límite se mudó de etapa.

## 2. Arquitectura del sistema

### 2.1. Servicios

Tabla 2: Servicios del sistema. Todos en Python; API con FastAPI, UI con Streamlit.

| Servicio        | Responsabilidad                                                                              | Tecnología               |
|-----------------|----------------------------------------------------------------------------------------------|--------------------------|
| orders-api      | Crea órdenes, las persiste y las encola                                                      | FastAPI + SQLite + Redis |
| estacion (×4–6) | Procesa lotes; una imagen única configurada por variables de entorno                         | Python + Redis + SQLite  |
| metrics-api     | Calcula espera, servicio y lead time; detecta el cuello de botella                           | FastAPI + SQLite + Redis |
| ui              | Tablero de operaciones: timeline, carga por réplica, alertas, histórico, creación de órdenes | Streamlit                |
| Redis           | Colas de trabajo y estado de réplicas                                                        | Redis 7                  |

Las cuatro estaciones son **un solo programa** desplegado cuatro (o más) veces: la etapa, sus colas de entrada/salida y su tiempo de ciclo llegan por variables de entorno (NOMBRE\_ETAPA, COLA\_ENTRADA, COLA\_SALIDA, TIEMPO\_CICLO). Escalar la etapa replicada es `docker compose up --scale envasado=3`: ninguna réplica sabe cuántas hermanas tiene ni necesita saberlo.

### 2.2. Nodos lógicos y su justificación

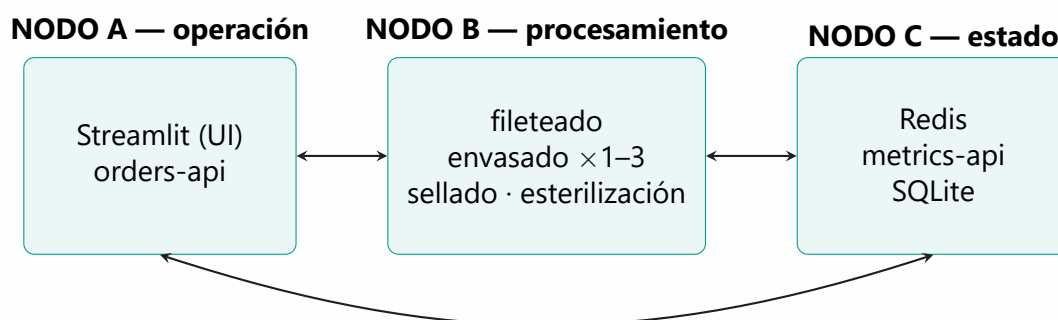


Figura 2: Los tres nodos lógicos y sus comunicaciones (HTTP/JSON y protocolo Redis).

La agrupación no es arbitraria: cada nodo reúne responsabilidades que **cambian de escala por motivos distintos**, el criterio clásico para particionar un sistema distribuido [1]. El nodo B es el único que se replica bajo carga (agregar capacidad de procesamiento

= agregar contenedores ahí). El nodo C concentra el estado compartido, que por ser compartido *no puede* duplicarse sin coordinación adicional. El nodo A expone el sistema al operador y escala con los usuarios, no con la producción.

**Por qué mantener dos o más nodos** (exigencia de la rúbrica): además de ser requisito del enunciado, separa dominios de falla y de recursos — la saturación de CPU de las estaciones (nodo B) no degrada la atención HTTP del operador (nodo A) ni la integridad del estado (nodo C); y permite dimensionar hardware distinto para cargas distintas (B necesita CPU, C necesita disco confiable). La sección §7 discute cómo se abordaría la resiliencia sobre esta misma división.

### 2.3. Flujo de una orden

Una orden nace en `orders-api` (`POST /ordenes`): se inserta en SQLite, se registra su evento `entraCola` y su `id` se empuja a `cola:fileteado` — todo dentro de una transacción, de modo que si Redis no responde no quedan “órdenes fantasma” guardadas pero nunca encoladas. Cada estación repite el mismo ciclo: reclama un `id` de su cola de entrada (`BRPOP`), registra `inicia`, simula el ciclo (`sleep(TIEMPO_CICLO)`), registra `termina` y `entraCola` de la etapa siguiente, y empuja el `id` a la cola siguiente. La última etapa marca la orden como `terminada`.

## 3. Decisiones de diseño

### 3.1. Balanceo por demanda: cola *pull*, no balanceador clásico

El enunciado sugiere “réplicas detrás de un balanceador”. Decidimos —y es una **decisión nuestra que defendemos aquí**— implementar el balanceo como **cola de trabajo compartida**: las réplicas *piden* un lote cuando quedan libres, en vez de recibirlo asignado por una pieza intermedia — el patrón de consumidores en competencia sobre un broker de mensajes [2].

1. Es lo único que produce balanceo **dinámico** real. Un round-robin le seguiría enviando lotes a una réplica lenta o atascada; con reparto por demanda, la réplica lenta simplemente pide menos veces. El experimento 4 (§4.2) lo muestra con datos: 11/12/7.
2. Resuelve la condición de carrera **con el mismo mecanismo**: la extracción atómica de la cola es a la vez el reparto y la garantía de exclusión (§3.2), sin una pieza extra que configurar ni un lock con expiración que se pueda vencer en el momento

equivocado.

3. El reparto emergente es proporcional a la capacidad real de cada réplica, sin conocer cuántas hay: `--scale envasado=3` funciona sin tocar configuración.

El “balanceador” del enunciado existe, pero es la cola: el punto único por el que pasa el trabajo y que decide quién procesa qué. Si se exigiera un balanceador HTTP clásico, un Nginx delante de las réplicas cubriría las consultas de estado; para el *trabajo*, el reparto por demanda es estrictamente superior en este caso.

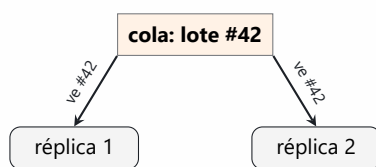
### 3.2. Coordinación y condición de carrera: provocarla antes de resolverla

El requisito de coordinación (“que dos réplicas no tomen la misma orden”) se abordó en dos pasos deliberados, ambos preservados en el historial de commits:

**Versión ingenua (commiteada rota a propósito).** Reclamar una orden en dos operaciones: mirar la cola (LRANGE) y después sacar el elemento (LREM). Entre ambas hay una ventana en la que otra réplica ve *la misma* orden. Con 3 réplicas y 200 órdenes, el experimento reprodujo órdenes procesadas dos veces — la evidencia de que el problema es real y no teórico.

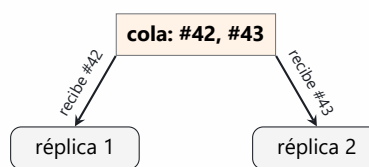
**Versión atómica.** BRPOP: extracción bloqueante en **una sola operación indivisible** [5]. Redis, al ser monohilo en la atención de comandos, garantiza que cada elemento se entrega a exactamente una réplica aunque haya varias esperando sobre la misma cola. Mismo experimento: **0 duplicados en 200 órdenes.**

Versión ingenua: mirar y sacar (2 pasos)



ambas procesan #42: duplicado

Versión atómica: BRPOP (1 paso)



cada lote a exactamente una réplica

Figura 3: La condición de carrera y su solución. A la izquierda, entre “mirar” y “sacar” hay una ventana en la que dos réplicas ven el mismo lote; a la derecha, la extracción es una única operación indivisible y la ventana no existe.

Se prefirió la operación atómica de cola por sobre un lock/contador distribuido (la alternativa que el enunciado menciona como ejemplo) porque elimina la ventana en vez de administrarla: no hay TTL que expire a mitad de un ciclo ni liberación de lock



que se pueda olvidar. El requisito del “lock/contador distribuido” está cubierto por el mismo Redis que serializa las extracciones.

### 3.3. Criterio de cuello de botella: espera, no servicio

El enunciado no define qué es un cuello de botella; el criterio es una **decisión nuestra** y es el argumento técnico central del proyecto:

El cuello de botella es la etapa con **mayor tiempo de espera promedio en una ventana móvil**, siempre que supere su umbral de advertencia. Si ninguna lo supera, no hay cuello (cuello: null).

Con los eventos de cada orden se definen, por etapa:

$$\text{espera} = t_{\text{inicia}} - t_{\text{entra\_cola}}, \quad \text{servicio} = t_{\text{termina}} - t_{\text{inicia}}, \quad \text{lead time} = t_{\text{fin}} - t_{\text{creada}}$$

Un tiempo de *servicio* alto solo dice que la tarea es larga; el atasco es trabajo llegando más rápido de lo que la etapa lo drena, y eso se manifiesta en la *espera* — la misma relación entre llegadas, capacidad y fila que formaliza la ley de Little [4] y que la teoría de las restricciones usa para localizar la limitante de un proceso [3]. La analogía cotidiana: en un supermercado, que un cajero sea lento no significa que haya atasco; el atasco es la *fila* que crece frente a su caja. El experimento 2 lo ilustra con datos (Figura 4): la etapa señalada como cuello (esterilización, ciclo 7 s) tiene un ciclo **más corto** que envasado (12 s) — un criterio por tiempo de servicio jamás la habría encontrado.

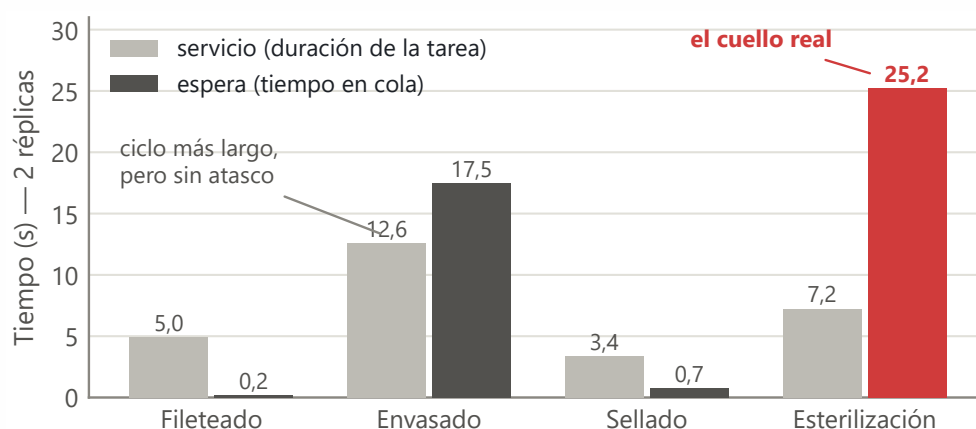


Figura 4: Servicio contra espera por etapa con 2 réplicas de envasado (experimento 2). Envasado tiene el ciclo más largo, pero la fila que crece — el cuello real — está en esterilización. Por eso el detector mide espera.



SQL ite en un volumen de Docker, con dos tablas: enderos(

1999, 2000, 2001, 2002, 2003, 2004, 2005, 2006, 2007, 2008, 2009, 2010, 2011, 2012, 2013, 2014, 2015, 2016, 2017, 2018, 2019, 2020, 2021, 2022, 2023, 2024, 2025, 2026, 2027, 2028, 2029, 2030, 2031, 2032, 2033, 2034, 2035, 2036, 2037, 2038, 2039, 2040, 2041, 2042, 2043, 2044, 2045, 2046, 2047, 2048, 2049, 2050, 2051, 2052, 2053, 2054, 2055, 2056, 2057, 2058, 2059, 2060, 2061, 2062, 2063, 2064, 2065, 2066, 2067, 2068, 2069, 2070, 2071, 2072, 2073, 2074, 2075, 2076, 2077, 2078, 2079, 2080, 2081, 2082, 2083, 2084, 2085, 2086, 2087, 2088, 2089, 2090, 2091, 2092, 2093, 2094, 2095, 2096, 2097, 2098, 2099, 2100, 2101, 2102, 2103, 2104, 2105, 2106, 2107, 2108, 2109, 2110, 2111, 2112, 2113, 2114, 2115, 2116, 2117, 2118, 2119, 2120, 2121, 2122, 2123, 2124, 2125, 2126, 2127, 2128, 2129, 2130, 2131, 2132, 2133, 2134, 2135, 2136, 2137, 2138, 2139, 2140, 2141, 2142, 2143, 2144, 2145, 2146, 2147, 2148, 2149, 2150, 2151, 2152, 2153, 2154, 2155, 2156, 2157, 2158, 2159, 2160, 2161, 2162, 2163, 2164, 2165, 2166, 2167, 2168, 2169, 2170, 2171, 2172, 2173, 2174, 2175, 2176, 2177, 2178, 2179, 2180, 2181, 2182, 2183, 2184, 2185, 2186, 2187, 2188, 2189, 2190, 2191, 2192, 2193, 2194, 2195, 2196, 2197, 2198, 2199, 2200, 2201, 2202, 2203, 2204, 2205, 2206, 2207, 2208, 2209, 2210, 2211, 2212, 2213, 2214, 2215, 2216, 2217, 2218, 2219, 2220, 2221, 2222, 2223, 2224, 2225, 2226, 2227, 2228, 2229, 2230, 2231, 2232, 2233, 2234, 2235, 2236, 2237, 2238, 2239, 2240, 2241, 2242, 2243, 2244, 2245, 2246, 2247, 2248, 2249, 2250, 2251, 2252, 2253, 2254, 2255, 2256, 2257, 2258, 2259, 2260, 2261, 2262, 2263, 2264, 2265, 2266, 2267, 2268, 2269, 2270, 2271, 2272, 2273, 2274, 2275, 2276, 2277, 2278, 2279, 2280, 2281, 2282, 2283, 2284, 2285, 2286, 2287, 2288, 2289, 2290, 2291, 2292, 2293, 2294, 2295, 2296, 2297, 2298, 2299, 2300, 2301, 2302, 2303, 2304, 2305, 2306, 2307, 2308, 2309, 2310, 2311, 2312, 2313, 2314, 2315, 2316, 2317, 2318, 2319, 2320, 2321, 2322, 2323, 2324, 2325, 2326, 2327, 2328, 2329, 2330, 2331, 2332, 2333, 2334, 2335, 2336, 2337, 2338, 2339, 2340, 2341, 2342, 2343, 2344, 2345, 2346, 2347, 2348, 2349, 2350, 2351, 2352, 2353, 2354, 2355, 2356, 2357, 2358, 2359, 2360, 2361, 2362, 2363, 2364, 2365, 2366, 2367, 2368, 2369, 2370, 2371, 2372, 2373, 2374, 2375, 2376, 2377, 2378, 2379, 2380, 2381, 2382, 2383, 2384, 2385, 2386, 2387, 2388, 2389, 2390, 2391, 2392, 2393, 2394, 2395, 2396, 2397, 2398, 2399, 2400, 2401, 2402, 2403, 2404, 2405, 2406, 2407, 2408, 2409, 2410, 2411, 2412, 2413, 2414, 2415, 2416, 2417, 2418, 2419, 2420, 2421, 2422, 2423, 2424, 2425, 2426, 2427, 2428, 2429, 2430, 2431, 2432, 2433, 2434, 2435, 2436, 2437, 2438, 2439, 2440, 2441, 2442, 2443, 2444, 2445, 2446, 2447, 2448, 2449, 2450, 2451, 2452, 2453, 2454, 2455, 2456, 2457, 2458, 2459, 2460, 2461, 2462, 2463, 2464, 2465, 2466, 2467, 2468, 2469, 2470, 2471, 2472, 2473, 2474, 2475, 2476, 2477, 2478, 2479, 2480, 2481, 2482, 2483, 2484, 2485, 2486, 2487, 2488, 2489, 2490, 2491, 2492, 2493, 2494, 2495, 2496, 2497, 2498, 2499, 2500, 2501, 2502, 2503, 2504, 2505, 2506, 2507, 2508, 2509, 2510, 2511, 2512, 2513, 2514, 2515, 2516, 2517, 2518, 2519, 2520, 2521, 2522, 2523, 2524, 2525, 2526, 2527, 2528, 2529, 2530, 2531, 2532, 2533, 2534, 2535, 2536, 2537, 2538, 2539, 2540, 2541, 2542, 2543, 2544, 2545, 2546, 2547, 2548, 2549, 2550, 2551, 2552, 2553, 2554, 2555, 2556, 2557, 2558, 2559, 2560, 2561, 2562, 2563, 2564, 2565, 2566, 2567, 2568, 2569, 2570, 2571, 2572, 2573, 2574, 2575, 2576, 2577, 2578, 2579, 2580, 2581, 2582, 2583, 2584, 2585, 2586, 2587, 2588, 2589, 2590, 2591, 2592, 2593, 2594, 2595, 2596, 2597, 2598, 2599, 2600, 2601, 2602, 2603, 2604, 2605, 2606, 2607, 2608, 2609, 2610, 2611, 2612, 2613, 2614, 2615, 2616, 2617, 2618, 2619, 2620, 2621, 2622, 2623, 2624, 2625, 2626, 2627, 2628, 2629, 2630, 2631, 2632, 2633, 2634, 2635, 2636, 2637, 2638, 2639, 2640, 2641, 2642, 2643, 2644, 2645, 2646, 2647, 2648, 2649, 2650, 2651, 2652, 2653, 2654, 2655, 2656, 2657, 2658, 2659, 2660, 2661, 2662, 2663, 2664, 2665, 2666, 2667, 2668, 2669, 2670, 2671, 2672, 2673, 2674, 2675, 2676, 2677, 2678, 2679, 2680, 26

El tablero (Streamlit nuevo

De la 1.ª a la 3.ª etapa

cuya capacidad queda *bajo* la línea de llegada acumula fila tarde o temprano — y con 2 réplicas envasado sube a 10 lotes/min: mejor, pero aún insuficiente, lo que anticipa los resultados. Datos crudos en experimentos/resultados/.

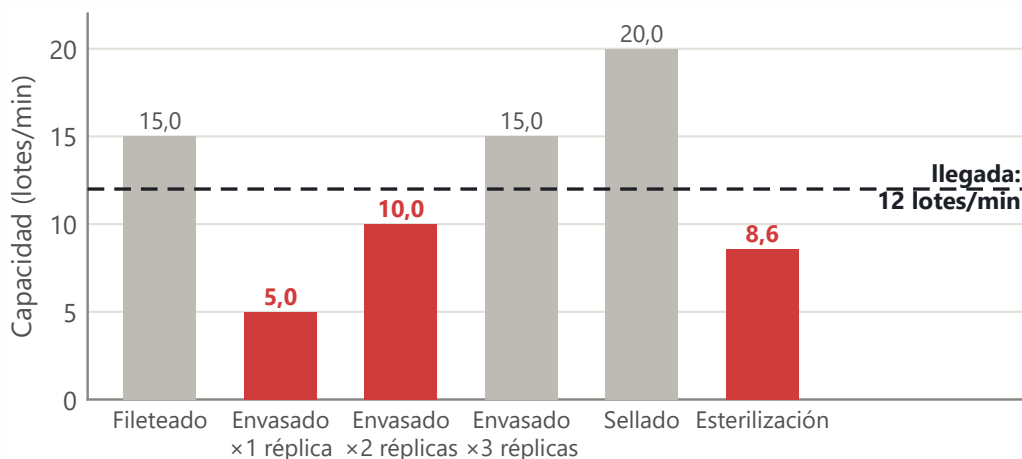


Figura 5: Capacidad de cada etapa (lotes/min) frente al ritmo de llegada del protocolo. Las etapas bajo la línea segmentada no alcanzan a drenar lo que les llega: ahí se forman las filas.

#### 4.1. Experimentos 1–3: el cuello se desplaza al escalar

Tabla 3: Resultado central: espera promedio por etapa (s) según réplicas de envasado.

| Réplicas | Fileteado | Envasado               | Sellado | Esterilización         | Lead time |
|----------|-----------|------------------------|---------|------------------------|-----------|
| 1        | 0,66      | <b>67,78 (crítico)</b> | 0,68    | 0,88                   | 82,9 s    |
| 2        | 0,18      | 17,52                  | 0,74    | <b>25,21 (crítico)</b> | 65,6 s    |
| 3        | 0,22      | 0,73                   | 0,89    | <b>34,50 (crítico)</b> | 62,0 s    |

- **1 réplica:** envasado acumula 14 lotes en cola y espera de 67,8 s; el resto de la línea, ociosa a ratos, espera < 1 s.
- **2 réplicas:** el reparto por demanda divide el trabajo (13/12 lotes por réplica) y el tiempo efectivo de envasado baja a ~6 s por lote. El cuello **se desplaza a esterilización** — la premisa del caso, demostrada con datos. Lead time: –21 %.
- **3 réplicas:** envasado queda con capacidad sobrada (espera 0,7 s), pero el lead time solo mejora otro 5 %: el límite ya no está ahí. La inversión correcta siguiente sería una segunda autoclave, no una tercera envasadora.

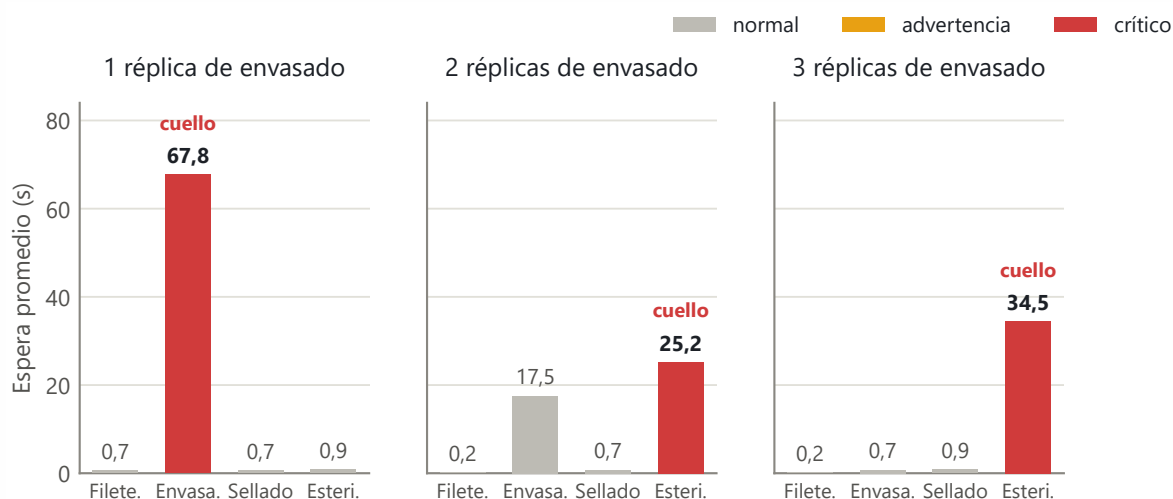


Figura 6: El resultado central, visto de una vez: la espera promedio por etapa según el número de réplicas de envasado. El cuello (barra roja) no desaparece al escalar: se desplaza de envasado a esterilización.

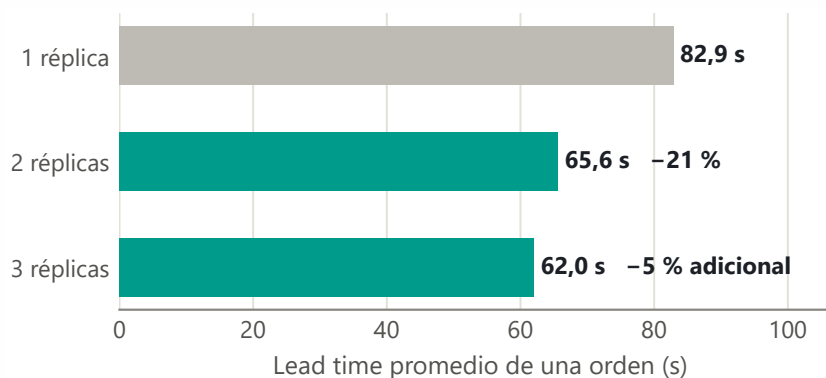


Figura 7: Lead time promedio de una orden (creación → terminada). La segunda réplica compra un 21 %; la tercera casi nada, porque el límite ya no está en envasado.

Los tiempos de servicio medidos (4,6–5,0 / 12,2–13,0 / 3,3–3,5 / 7,3 s) coinciden con los ciclos configurados (4/12/3/7 s), lo que valida la instrumentación.

#### 4.2. Experimento 4: balanceo dinámico con una réplica lenta

Dos réplicas normales más una configurada al doble de ciclo, sobre la misma cola, 30 lotes: la lenta procesó **7** contra **11 y 12** de las normales (Figura 8). Un round-robin habría repartido 10/10/10 sin importar la velocidad. Nadie asignó ese reparto: emergió de que cada réplica reclama trabajo solo al quedar libre.

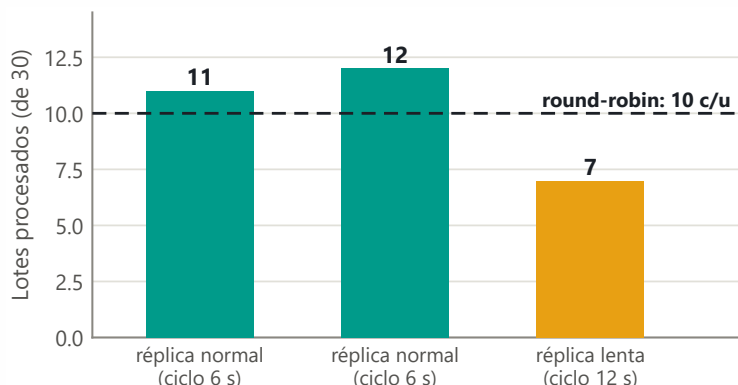


Figura 8: Balanceo dinámico con una réplica al doble de ciclo: el reparto se adapta solo a la capacidad real de cada una, sin configuración ni coordinación explícita.

### 4.3. Experimento 5: saturación y recuperación automática

Ráfaga de 25 órdenes de golpe (envasado  $\times 2$ ), observando `/metricas/cuello` cada 15 s: a los 17 s la primera etapa pasa a advertencia y a los 32 s a crítico — **sola**. La Figura 9 muestra la película completa: la congestión recorre la línea como una ola (fileteado  $\rightarrow$  envasado  $\rightarrow$  esterilización) y a los **278 s** las cuatro etapas están de vuelta en normal **sin intervención alguna**: las esperas viejas salieron de la ventana móvil. Detalle en `experimentos/resultados/exp5-saturacion.txt`.

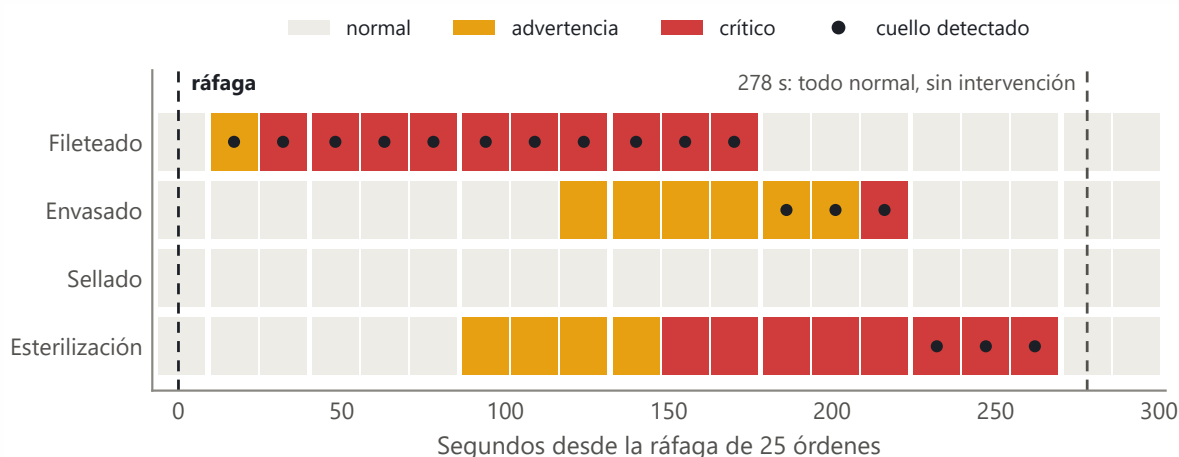


Figura 9: Experimento 5 como línea de tiempo de estados: cada fila es una etapa y cada celda una sonda de `/metricas/cuello`. La congestión avanza por la línea y el sistema vuelve a normal por sí solo.

La ráfaga deja además una lección de operación: con todo inyectado a la vez, el cuello detectado inicialmente es la *primera* etapa (ahí se apila la cola), no la de mayor ciclo.

El cuello depende del patrón de llegada, no solo de los tiempos de proceso — por eso el protocolo de los experimentos 1–3 inyecta a ritmo constante.

## 5. Manejo de errores

Códigos verificados contra el sistema corriendo: 201 (creación), 422 (cantidad  $\leq 0$ , producto vacío, filtro inválido — nunca 500), 404 (orden inexistente), 503 (dependencia caída, con mensaje en castellano). La UI nunca muestra un traceback: ante cualquier falla consulta la salud de los tres servicios y **nombra al que realmente falta** — si Redis cae, los demás siguen vivos y culpar al que responde lento enviaría a buscar el error al lugar equivocado.

El hallazgo técnico de esta parte: con Redis detenido, los timeouts de socket del cliente **no bastan**, porque el cuelgue ocurre antes, en la resolución DNS (Docker elimina el nombre del contenedor detenido y `getaddrinfo` tarda  $\sim 45$  s en rendirse — medido). La solución fue un **plazo duro**: la consulta a Redis corre en un hilo aparte y se espera su resultado a lo más 2 s; si no llega, se responde 503 de inmediato. Resultado medido: `POST /ordenes` con Redis caído pasó de colgarse 46 s a responder 503 en **2,1 s**, y `/metricas` responde 200 en 2,0 s con `en_cola: null` (un `null` honesto; un 0 sería mentira).

Todos los servicios exponen `GET /salud` y tienen `healthcheck` en el `compose`. El `healthcheck` mide *vida*, no dependencias: un servicio que responde 503 explicándose está sano — marcar `unhealthy` servicios vivos habría provocado reinicios en cascada de contenedores que no tienen nada malo.

## 6. Reproducibilidad y despliegue

```
git clone <repositorio> && cd <carpeta>
docker compose up -d --build
# tablero en http://localhost:8501
```

Sin pasos manuales, sin instalar nada más que Docker [7]. La prueba se ejecutó de verdad: clon en carpeta vacía → 8/8 contenedores `healthy` → orden creada por API recorre las cuatro etapas hasta `terminada` → tablero arriba. Toda la configuración tiene defaults funcionales (`.env.example` documenta cada variable; no es obligatorio crearlo). El README cubre instalación, uso, configuración, experimentos y diagnóstico de fallas comunes.



## 7. Resiliencia: cómo se abordaría

El enunciado no exige tolerancia a fallos y este proyecto, deliberadamente, **detecta y explica** las fallas en vez de enmascararlas (sin reintentos, sin circuit breaker). Si se necesitara operar esto en producción, la arquitectura ya deja los puntos de corte correctos:

- **Estaciones (nodo B):** ya son resilientes en lo esencial — sin estado propio, reconectan solas (bucle de reintento del worker) y pueden multiplicarse o morir sin corromper nada. Lo único pendiente sería re-encolar el lote en curso si una réplica muere a mitad de ciclo (hoy ese lote se pierde de la cola; el patrón estándar es BLMOVE a una cola “en proceso” por réplica, devuelta a la cola principal por un barrendero al detectar un latido vencido).
- **Redis (nodo C):** es el punto único de falla del reparto. El camino estándar es Redis Sentinel (réplica + failover automático) o un broker con confirmaciones (el bonus Kafka apuntaba ahí): con *acks*, un mensaje no confirmado vuelve a entregarse solo.
- **SQLite (nodo C):** suficiente para un nodo; con más escritores o failover se migraría a MySQL/PostgreSQL gestionado, cambiando solo el módulo `bd.py` de cada servicio.
- **APIs y UI (nodo A):** sin estado; con dos instancias y un balanceador HTTP delante quedarían en alta disponibilidad. Los healthchecks ya existentes son el insumo de ese balanceador.

La división en nodos de §2.2 es exactamente la que hace estos upgrades locales: cada mejora toca un nodo sin rediseñar los demás.

## 8. Análisis crítico de dificultades

1. **El entorno costó más que el primer código.** La virtualización venía deshabilitada en BIOS (SVM Mode); `wsl --install` no instaló ninguna distribución (hubo que pedir Ubuntu aparte); y Docker Desktop caía en cascada por sockets huérfanos de un arranque fallido que Windows no podía borrar, sembrando cada caída el bloqueo de la siguiente. Lección: el “hola mundo” de infraestructura es parte del proyecto y hay que presupuestarlo.
2. **La condición de carrera hubo que fabricarla para verla.** Con el reclamo ingenuo, la ventana entre mirar y sacar es de microsegundos: sin duplicados no hay demostración. Se ensanchó con una espera artificial configurable



(DEMORA\_INGENUA) hasta reproducir duplicados de forma confiable, y ambas versiones quedaron commiteadas — entender el problema antes de resolverlo.

3. **“Le puse timeout” no es verificable hasta medir el camino completo de la falla.** El cuelgue con Redis caído no estaba en el socket sino en el DNS (46 s medidos), que el timeout configurado no cubría. Hubo que aislar la falla por capas (resolución → conexión → lectura) y acotar la operación completa con un plazo duro.
4. **Al acelerar una simulación se puede medir otra cosa.** El experimento de la réplica lenta con ciclos de  $\sim 1$  s dio un reparto casi parejo (21/20/19): a ese ritmo, el costo por lote lo dominaba la escritura de eventos en SQLite (seis procesos compitiendo por el lock sobre el volumen), no el ciclo simulado. Con ciclos de 6/12 s el overhead se vuelve despreciable y el resultado es el esperado (11/12/7). El overhead constante no escala con la simulación.
5. **El cuello de botella depende del patrón de llegada.** Inyectar lotes de golpe apila todo en la primera cola y el sistema señala —correctamente— a fileteado. Entender que eso no era un bug sino una propiedad de las colas cambió el protocolo experimental y el guion de la demo: ritmo constante para responder “¿qué etapa limita la producción?”.

## 9. Supuestos declarados

1. Los tiempos de ciclo (4/12/3/7 s) son inventados, elegidos para que el desplazamiento del cuello sea demostrable en una demo corta. No representan una planta real; el escenario es ficticio.
2. “Procesar un lote” se simula esperando el tiempo de ciclo; no hay trabajo real.
3. Un lote representa una cantidad de latas del mismo formato; no se modelan latas individuales.
4. El enunciado no fija qué persistir: decidimos órdenes y eventos, con las métricas siempre derivadas.
5. El enunciado no define el criterio de cuello de botella ni umbrales: el criterio de \$3.3 es decisión nuestra.

## 10. Conclusiones

El sistema cumple los requisitos del caso de punta a punta y sus afirmaciones están respaldadas por mediciones reproducibles: el reparto por demanda balancea sin conocer las réplicas (11/12/7 con una réplica lenta), la extracción atómica elimina la condición de carrera (0 duplicados en 200 órdenes, contra duplicados reproducibles de la versión ingenua), el criterio de espera en ventana móvil detecta el cuello de botella correcto incluso cuando no es la etapa más lenta, y se recupera solo al bajar la carga. El resultado central — el cuello se desplaza de envasado a esterilización al escalar de 1 a 2 réplicas, y una tercera ya no compra casi nada — es la demostración empírica de que en una línea en serie *el cuello de botella no se elimina: se muda* [3], y de que medir espera (no servicio) es lo que permite verlo en vivo.

## Referencias

- [1] M. van Steen y A. S. Tanenbaum, *Distributed Systems*, 4.<sup>a</sup> ed. Maarten van Steen, 2023. Disponible en <https://www.distributed-systems.net>.
- [2] M. Kleppmann, *Designing Data-Intensive Applications*. Sebastopol, CA: O'Reilly Media, 2017.
- [3] E. M. Goldratt y J. Cox, *La Meta: un proceso de mejora continua*, 3.<sup>a</sup> ed. Great Barrington, MA: North River Press, 2004.
- [4] J. D. C. Little, "A proof for the queuing formula  $L = \lambda W$ ", *Operations Research*, vol. 9, n.º 3, pp. 383–387, 1961.
- [5] Redis Ltd., "BRPOP — Redis command reference", documentación oficial de Redis 7, 2024. Disponible en <https://redis.io/docs/latest/commands/brpop/>.
- [6] SQLite Consortium, "Write-Ahead Logging", documentación oficial de SQLite, 2024. Disponible en <https://www.sqlite.org/wal.html>.
- [7] Docker Inc., "Docker Compose overview", documentación oficial de Docker, 2024. Disponible en <https://docs.docker.com/compose/>.



## A. Contratos completos de las APIs

Los tres servicios hablan REST sobre JSON. Todos exponen GET /salud → 200 {".estado": ".ok"}}, que es lo que consultan los *healthchecks* de Docker Compose. El healthcheck mide **vida del proceso**, no disponibilidad de sus dependencias: un servicio que responde 503 explicando que Redis no está sigue estando sano.

### A.1. orders-api (puerto 8000)

#### POST /ordenes — crear una orden de producción

Entrada: { "producto": "jurel-425g", "cantidad": 10 }

Respuesta 201:

```
{
  "id": 1,
  "producto": "jurel-425g",
  "cantidad": 10,
  "estado": "en_proceso",
  "creada_en": "2026-08-21T17:30:00+00:00",
  "terminada_en": null
}
```

| Campo    | Regla de validación                        |
|----------|--------------------------------------------|
| producto | string no vacío (tampoco solo espacios)    |
| cantidad | entero <b>mayor que 0</b> — latas del lote |

Devuelve **422** si la entrada no cumple las reglas: nunca 500 y nunca 201 con datos inválidos. Como efecto colateral, el id se encola en cola:fileteado (Redis).

#### GET /ordenes/{id} y GET /ordenes?estado=...

- GET /ordenes/{id} → **200** con la orden completa, incluido terminada\_en (null si sigue en proceso). **404** si el id no existe.
- GET /ordenes?estado=<en\_proceso|terminada> → **200** con la lista, filtrada si viene el parámetro. **422** si estado trae un valor fuera del catálogo.

### A.2. estacion (una imagen, cuatro despliegues)

Toda la configuración entra por variables de entorno; **ningún** valor de etapa está escrito en el código:

NOMBRE\_ETAPA=envasado



```
COLA_ENTRADA=cola:envasado
COLA_SALIDA=cola:sellado
TIEMPO_CICLO=12
```

#### GET /estado

```
{
  "etapa": "envasado",
  "replica_id": "envasado-a1b2c3",
  "procesadas": 17,
  "ocupada": true,
  "enCola": 4
}
```

enCola es el largo actual de la cola de entrada (LLEN). replica\_id se genera al arrancar — nombre de etapa más sufijo único — y aparece también en cada línea de log, que es lo que permite atribuir cada lote procesado a una réplica concreta en el experimento 4.

### A.3. metrics-api (puerto 8001)

#### GET /metricas

Por etapa, calculado siempre desde la tabla eventos:

```
{
  "ventana_s": 60,
  "etapas": {
    "envasado": {
      "espera_promedio_s": 8.2,
      "servicio_promedio_s": 12.1,
      "enCola": 4,
      "procesadas": 17
    }
  },
  "lead_time_promedio_s": 31.4
}
```

#### GET /metricas/cuello

```
{
  "cuello": "esterilizacion",
  "estados": { "fileteado": "normal", "envasado": "normal",
    "sellado": "normal", "esterilizacion": "critico" },
  "razon": "espera promedio 14.3 s en la ventana de 60 s,"
}
```



```
sobre el umbral critico"
}
```

cuello es null cuando ninguna etapa supera su umbral de advertencia. El campo razon viene del servicio, no de la interfaz: el diagnóstico y su justificación salen del mismo lugar.

**GET /metricas/historico**

Serie temporal muestreada para los gráficos de la interfaz — la rúbrica exige histórico además de tiempo real:

```
{ "puntos": [ { "timestamp": "...", "etapa": "envasado",
                "espera_promedio_s": 8.2, "enCola": 4 } ] }
```

#### A.4. Códigos HTTP en uso

| Código | Cuándo              | Verificado con                                              |
|--------|---------------------|-------------------------------------------------------------|
| 201    | Orden creada        | POST /ordenes con cuerpo válido                             |
| 422    | Entrada inválida    | cantidad $\leq$ 0, producto vacío, estado fuera de catálogo |
| 404    | Recurso inexistente | GET /ordenes/99999                                          |
| 503    | Dependencia caída   | docker compose stop redis y luego POST /ordenes             |

## B. Esquema de datos y momentos de escritura

Dos tablas, y nada más. Todo lo demás se **calcula** desde eventos: el dato guardado dos veces es el dato que termina contradiciéndose.

```
CREATE TABLE ordenes (
  id          INTEGER PRIMARY KEY AUTOINCREMENT,
  producto    TEXT      NOT NULL,
  cantidad    INTEGER NOT NULL CHECK (cantidad > 0),
  estado      TEXT      NOT NULL DEFAULT 'en_proceso',
  creada_en   TEXT      NOT NULL,    -- ISO 8601 UTC
  terminada_en TEXT      -- NULL hasta terminar
);

CREATE TABLE eventos (
  id          INTEGER PRIMARY KEY AUTOINCREMENT,
  orden_id    INTEGER NOT NULL REFERENCES ordenes(id),
  etapa       TEXT      NOT NULL,    -- fileteado / envasado /
                                     -- sellado / esterilizacion
  replica_id  TEXT      NOT NULL,    -- que replica exacta lo hizo
  tipo        TEXT      NOT NULL,    -- entra_cola / inicia / termina
  timestamp   TEXT      NOT NULL    -- ISO 8601 UTC
);
```

| Momento                                | Quién escribe                      | Evento     |
|----------------------------------------|------------------------------------|------------|
| La orden entra a una cola              | orders-api, o la estación anterior | entra_cola |
| Una réplica la saca de la cola (BRPOP) | la réplica                         | inicia     |
| La réplica termina su ciclo            | la réplica                         | termina    |

Una orden que recorre las cuatro etapas genera **12 eventos**, tres por etapa. El último termina de esterilización además marca ordenes.estado = 'terminada' y escribe terminada\_en.

Los estados de una orden son solo dos, y es deliberado: en\_proceso desde su creación y terminada al salir de esterilización. «En qué etapa va» **no** es un estado guardado — se deriva del último evento de esa orden. Guardarlo sería mantener dos versiones de la misma verdad.

De aquí salen las tres métricas del informe:

- **Espera** en una etapa:  $t(\text{inicia}) - t(\text{entra\_cola})$ .



- **Servicio** en una etapa:  $t(\text{termina}) - t(\text{inicia})$ .
- **Lead time** de la orden:  $t(\text{terminada\_en}) - t(\text{creada\_en})$ .

Ninguna de las tres se almacena. Que los servicios medidos coincidan con los ciclos configurados (4,8 / 12,4 / 3,4 / 7,3 s contra 4 / 12 / 3 / 7) es lo que valida la instrumentación contra sí misma.

## C. Capturas del tablero y verificación de errores

Las tres capturas se tomaron sobre el sistema corriendo con `docker compose up -d --scale envasado=2`, inyectando carga real por `POST /ordenes`. El color marca **importancia**, no identidad: lo normal es gris y verde, y el color fuerte queda reservado para advertencia y crítico, siempre acompañado de ícono y texto.

**Una aclaración necesaria.** En estas capturas la etapa señalada como cuello es **fileteado**, no esterilización como en los experimentos del §4. No es una contradicción, y vale la pena explicarlo porque es la primera pregunta que aparece al mirirlas: aquí los lotes se inyectaron **en ráfaga** para forzar los estados de advertencia y crítico rápidamente, y con llegadas en ráfaga la fila más larga se forma en la primera etapa de la línea, que es donde entran. El detector responde correctamente a lo que está pasando. Los experimentos del cuerpo usan llegadas **espaciadas** —una cada 5 s— que es el régimen en el que la pregunta del caso tiene sentido, y ahí el cuello aparece donde la aritmética de capacidad predice.

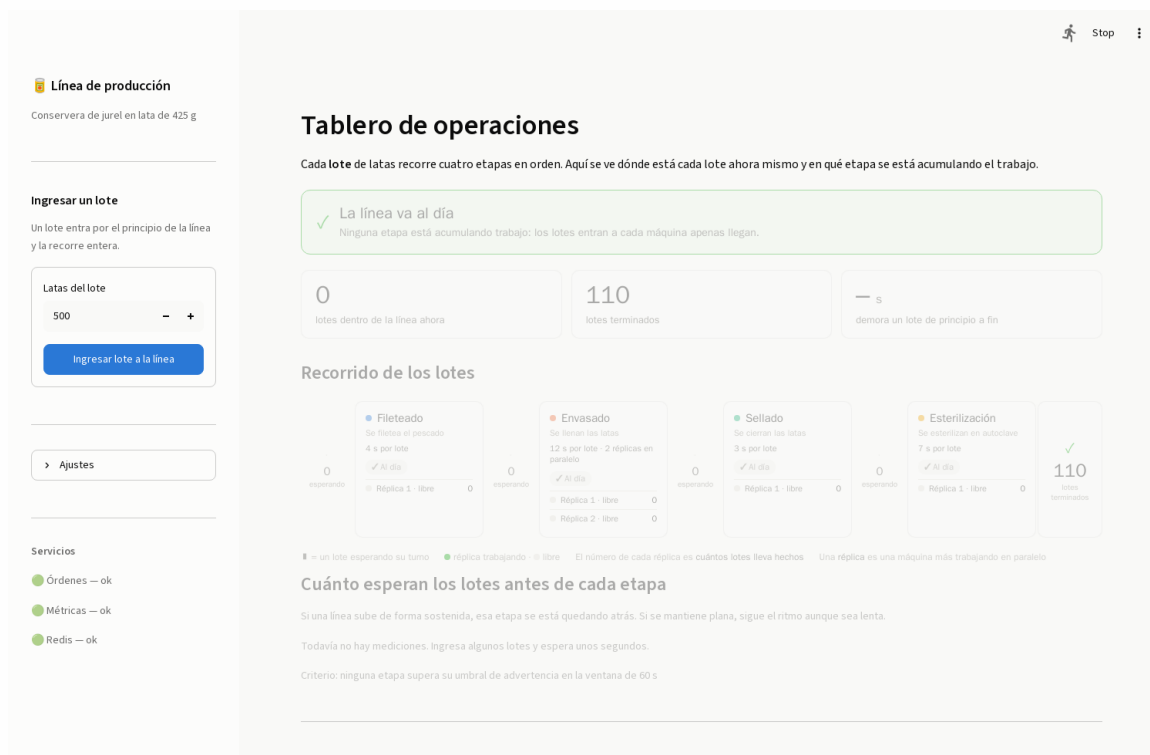


Figura 10: Línea al día. Ninguna etapa supera su umbral de advertencia; el cartel superior lo dice explícitamente y las cuatro etapas aparecen «Al día».



Figura 11: Estado de **advertencia** tras inyectar 14 lotes espaciados 2 s. La espera en fileteado llegó a 6,7 s contra su ciclo de 4 s: pasó el umbral de advertencia ( $1,5\times$ ) y no el crítico ( $3\times$ ). El cartel nombra la etapa, dice cuántos lotes hacen cola y da la cifra; la etapa aparece marcada «Acumulando» y su curva de espera empieza a subir.



Figura 12: Estado **crítico** tras una ráfaga de 25 lotes de golpe. La espera en fileteado subió a 12,5 s contra su ciclo de 4 s, sobre el umbral crítico (3×); la etapa queda marcada «Atascada» con 25 lotes en cola. El diagnóstico y su razón vienen de /metricas/cuello, no de la interfaz: el tablero solo lo muestra.

### C.1. Verificación de los códigos HTTP y de la caída de Redis

El cuarto estado —Redis caído— se documenta con la traza medida contra el sistema corriendo en vez de con una captura, porque es donde la evidencia realmente está: los tiempos y los códigos son verificables y una imagen del tablero no los muestra. La sesión completa, tal como salió:

```
### CON REDIS ARRIBA
POST /ordenes          -> HTTP 201   en 0.013184s
GET  /salud (ordenes)  -> HTTP 200   en 0.003583s

### VALIDACION DE ENTRADA
POST cantidad=0        -> HTTP 422
POST producto vacio    -> HTTP 422
GET  /ordenes/99999    -> HTTP 404
GET  /ordenes?estado=xxx -> HTTP 422

### DETENIENDO REDIS   (docker compose stop redis)
```



```
POST /ordenes          -> HTTP 503   en 2.005218s
  {"detail": "sin conexion con el servicio de coordinacion (Redis);
               intente de nuevo"}
GET  /salud (ordenes)   -> HTTP 200   en 0.003470s
GET  /metricas/cuello   -> HTTP 200   en 2.011873s

### LEVANTANDO REDIS    (docker compose start redis)
POST /ordenes          -> HTTP 201   en 0.012831s
```

Tres cosas que esta traza prueba y que son el punto del §7:

1. **El plazo duro funciona.** El 503 vuelve en **2,005 s**, no en los 46 s que medimos antes de descubrir que el cuelgue estaba en la resolución DNS y no en el socket. El `PLAZO_REDIS_S` configurado es 2 s y se cumple con cinco milésimas de margen.
2. **El mensaje nombra al culpable real**, en castellano y sin *traceback*: «sin conexión con el servicio de coordinación (Redis)». La interfaz muestra ese mismo texto, que viene del servicio.
3. **El healthcheck sigue en 200 y en 3 ms.** El servicio de órdenes está vivo: sabe que le falta una dependencia y lo dice. Si el healthcheck comprobara dependencias, Docker habría matado y reiniciado un contenedor que funcionaba perfectamente.

Al volver a levantar Redis, la creación de órdenes responde 201 en 13 ms sin reiniciar nada: la línea continúa donde quedó, porque las estaciones no tienen estado propio y reconectan solas.

## D. Salidas crudas de los experimentos

Todo lo que sigue está versionado en `experimentos/resultados/` y se reproduce con los guiones de `experimentos/`. Las figuras del cuerpo del informe se generan desde estos mismos archivos con `informe/figuras/generar_figuras.py`: no hay ningún número transcrito a mano entre la medición y el gráfico.

### D.1. Experimentos 1–3: el cuello se desplaza al escalar

```
--- exp1-ensvasado1-cuello.json ---
{"cuello": "ensvasado",
 "estados": {"fileteado": "normal", "ensvasado": "critico",
             "sellado": "normal", "esterilizacion": "normal"},
 "razon": "espera promedio 67.78 s en la ventana de 60 s,
          sobre el umbral critico (12 s de ciclo x 3)",
 "ventana_s": 60.0}

--- exp2-ensvasado2-cuello.json ---
{"cuello": "esterilizacion",
 "estados": {"fileteado": "normal", "ensvasado": "normal",
             "sellado": "normal", "esterilizacion": "critico"},
 "razon": "espera promedio 25.21 s en la ventana de 60 s,
          sobre el umbral critico (7 s de ciclo x 3)",
 "ventana_s": 60.0}

--- exp2-ensvasado2-metricas.json ---
{"ventana_s": 60.0, "etapas": {
  "fileteado": {"espera_promedio_s": 0.18, "servicio_promedio_s": 4.96,
               "enCola": 0, "procesadas": 31},
  "ensvasado": {"espera_promedio_s": 17.52, "servicio_promedio_s": 12.58,
               "enCola": 4, "procesadas": 25},
  "sellado": {"espera_promedio_s": 0.74, "servicio_promedio_s": 3.38,
              "enCola": 0, "procesadas": 24},
  "esterilizacion": {"espera_promedio_s": 25.21, "servicio_promedio_s": 7.25,
                    "enCola": 4, "procesadas": 22}}}

--- exp3-ensvasado3-cuello.json ---
{"cuello": "esterilizacion",
 "estados": {"fileteado": "normal", "ensvasado": "normal",
             "sellado": "normal", "esterilizacion": "critico"},
 "razon": "espera promedio 34.5 s en la ventana de 60 s,
          sobre el umbral critico (7 s de ciclo x 3)",
```

```
"ventana_s":60.0}

--- exp3-enzasado3-metricas.json ---
{"ventana_s":60.0,"etapas":{
  "fileteado":    {"espera_promedio_s":0.22,"servicio_promedio_s":4.59,
                  "en_cola":0,"procesadas":31},
  "enzasado":     {"espera_promedio_s":0.73,"servicio_promedio_s":12.24,
                  "en_cola":0,"procesadas":29},
  "sellado":      {"espera_promedio_s":0.89,"servicio_promedio_s":3.33,
                  "en_cola":0,"procesadas":28},
  "esterilizacion":{"espera_promedio_s":34.5,"servicio_promedio_s":7.3,
                  "en_cola":8,"procesadas":26}}}
```

**Limitación conocida.** El archivo exp1-enzasado1-metricas.json quedó **vacío** al capturarlo: la consulta se hizo después de que la ventana móvil de 60 s ya se había vaciado. El diagnóstico de cuello de ese experimento (exp1-enzasado1-cuello.json) sí quedó guardado, y es de donde sale la cifra de 67,8 s de espera. Los valores por etapa del experimento 1 en el cuerpo del informe provienen de esa corrida registrada en la bitácora de la Fase 14, no de un archivo crudo. Se declara aquí en vez de rehacerlo silenciosamente.

## D.2. Experimento 4: balanceo con una réplica lenta

```
--- exp4-replica-lenta.txt ---
enzasado-88d690e0069d      11
enzasado-b3573cf68ffd      12
enzasado-lento-8550b032c252  7

ciclos: enzasado=6s lento=12s, N=30 ordenes directas a cola:enzasado
esperado round-robin: 10/10/10 - esperado por demanda: ~12/12/6
```

El reparto medido fue 11/12/7 contra el 10/10/10 que habría dado un round-robin. La réplica lenta procesó un 40 % menos que las rápidas sin que nadie programara esa proporción: emergió de que cada réplica reclama un lote solo cuando queda libre.

## D.3. Experimento 5: saturación y recuperación automática

Ráfaga de 25 órdenes de golpe, sondeando /metricas/cuello cada 15 s.

```
--- exp5-saturacion.txt (extracto) ---
t=1s   cuello=None      fileteado:normal      enzasado:normal
t=17s  cuello=fileteado  fileteado:advertencia enzasado:normal
t=32s  cuello=fileteado  fileteado:critico     enzasado:normal
```



```
t=48s    cuello=fileteado    fileteado:critico    envasado:normal
t=63s    cuello=fileteado    fileteado:critico    envasado:advertencia
...
t=278s   cuello=None        todas:normal
```

La congestión recorre la línea como una ola: aparece primero en fileteado, que es donde entran los lotes, y se propaga aguas abajo. A los 278 s todo vuelve a normal **sin intervención**, porque las esperas antiguas salieron de la ventana móvil de 60 s. El sistema no «se arregla»: deja de estar congestionado y la métrica lo refleja.

## E. Historial de commits

El repositorio es <https://github.com/CesarOlivares/Estructura-de-computadores-entrega-3>. El trabajo está repartido entre los dos integrantes y distribuido en el tiempo, no concentrado en una sesión final.

| Autor                 | Commits   | Aporte principal                                           |
|-----------------------|-----------|------------------------------------------------------------|
| César Olivares        | 27        | Servicios, Redis, persistencia, métricas, informe          |
| Simón García-Huidobro | 15        | Tablero Streamlit, demo de la carrera, lanzadores, defensa |
| <b>Total</b>          | <b>42</b> | 21–25 de agosto de 2026                                    |

Los tres primeros commits son cargas por la interfaz web de GitHub, anteriores a fijar la convención del equipo. Desde `chore: initialize repository` en adelante se usa Conventional Commits, con un commit por fase verificada.

|         |            |       |                                                      |
|---------|------------|-------|------------------------------------------------------|
| 7a9d44b | 2026-08-21 | Cesar | Add files via upload                                 |
| b05cb63 | 2026-08-21 | Cesar | Add files via upload                                 |
| fba5747 | 2026-08-21 | Cesar | Add files via upload                                 |
| 1f3b57c | 2026-08-21 | Cesar | chore: initialize repository                         |
| bb87555 | 2026-08-21 | Cesar | demo: throwaway queue prototype                      |
| 176a0b1 | 2026-08-21 | Cesar | docs: close phase 0 environment log                  |
| 4330ff4 | 2026-08-21 | Cesar | docs: define api contracts and data model            |
| 8253914 | 2026-08-21 | Cesar | feat: minimal orders api with validation             |
| 6398e77 | 2026-08-21 | Cesar | feat: add redis queue and healthcheck                |
| ec6c211 | 2026-08-21 | Cesar | feat: station worker consuming orders                |
| 05efcc9 | 2026-08-21 | Cesar | feat: reproduce race condition with naive claiming   |
| 3f1c79b | 2026-08-21 | Cesar | feat: implement atomic order claiming                |
| c9c6fc3 | 2026-08-21 | Cesar | test: verify no duplicate processing under load      |
| d178cec | 2026-08-21 | Cesar | feat: scale stations and expose replica identity     |
| e435d42 | 2026-08-21 | Cesar | feat: dynamic load balancing with configurable times |
| 1c8ce5a | 2026-08-21 | Cesar | feat: persist orders and events in sqlite            |
| 28db438 | 2026-08-21 | Cesar | feat: metrics service with lead time breakdown       |
| b77b6a1 | 2026-08-21 | Cesar | feat: bottleneck detection with sliding window       |
| c937045 | 2026-08-21 | Simon | feat: streamlit operations dashboard                 |
| 3e9abf0 | 2026-08-21 | Simon | docs: readme with setup and run instructions         |
| 0b74b9b | 2026-08-22 | Simon | docs: align phase 11 log with team template          |
| f3d8d74 | 2026-08-22 | Cesar | feat: error handling and http status codes           |
| 44181d2 | 2026-08-22 | Cesar | docs: env example and reproducibility notes          |

|         |            |       |                                                        |
|---------|------------|-------|--------------------------------------------------------|
| 51858f2 | 2026-08-22 | Cesar | docs: readme verified on clean clone                   |
| 02bfe33 | 2026-08-22 | Cesar | docs: experiment results for 1, 2 and 3 replicas       |
| 7182916 | 2026-08-22 | Cesar | docs: architecture decisions and assumptions           |
| abfb9bb | 2026-08-23 | Simon | fix: count processed orders per stage                  |
| 1d0d90c | 2026-08-23 | Simon | refactor: extract claim strategies into shared module  |
| 5d4b125 | 2026-08-23 | Simon | feat: race condition demo in the dashboard             |
| 4fbcd17 | 2026-08-23 | Simon | feat: one-click launcher for windows, macos, linux     |
| d99d7ac | 2026-08-23 | Simon | docs: phase 16 log, readme and defence guide           |
| 25e84cb | 2026-08-23 | Cesar | chore: tidy repo layout for delivery                   |
| e060912 | 2026-08-23 | Cesar | docs: report with charts, diagrams and bibliography    |
| 08c47c3 | 2026-08-23 | Cesar | docs: defense slides with charts, exported to pdf      |
| ce6a19d | 2026-08-23 | Cesar | docs: demo slide covers the race comparison            |
| 7ed4fe4 | 2026-08-25 | Simon | chore: move course material into enunciado/            |
| 9d1ae9f | 2026-08-25 | Simon | chore: drop the internal working plan                  |
| 603c0a5 | 2026-08-25 | Simon | docs: readme points to the deliverables                |
| 669762a | 2026-08-25 | Simon | docs: slides drop topic numbering, compile on Overleaf |
| fff994f | 2026-08-25 | Simon | docs: second format for the defence slides             |
| 86cc321 | 2026-08-25 | Simon | fix: slides embed a font dvipdfmx can write            |
| 95b4894 | 2026-08-25 | Simon | docs: both slide PDFs compiled and current             |

La condición de carrera está en el historial como dos commits consecutivos: 05efcc9 la reproduce con el reclamo ingenuo y 3f1c79b la elimina con extracción atómica. No se resolvió el problema antes de tenerlo.

## F. Bitácoras por fase

El proyecto se organizó en fases; cada una deja una bitácora en docs/fases/ escrita al cerrarla, con objetivo, qué se logró, dificultades encontradas y pendientes. Son el insumo directo del análisis crítico del §8 de este informe.

| Archivo                      | Fase                                          | Estado        |
|------------------------------|-----------------------------------------------|---------------|
| fase-0-entorno.md            | Entorno y repositorio                         | Cerrada 21/08 |
| fase-2-diseno.md             | Diseño en papel                               | Diseño listo  |
| fase-3-orders-api.md         | orders-api mínima en Docker                   | Cerrada 21/08 |
| fase-4-redis.md              | Redis y la cola                               | Cerrada 21/08 |
| fase-5-estacion.md           | Una estación consume                          | Cerrada 21/08 |
| fase-6-carrera.md            | Condición de carrera: provocarla y resolverla | Cerrada 21/08 |
| fase-7-balanceo.md           | Balanceo dinámico con N réplicas              | Cerrada 21/08 |
| fase-8-persistencia.md       | Persistencia                                  | Cerrada 21/08 |
| fase-9-metricas.md           | Métricas                                      | Cerrada 21/08 |
| fase-10-cuello-de-botella.md | Cuello de botella y estados                   | Cerrada 21/08 |
| fase-11-ui.md                | Interfaz Streamlit                            | Cerrada 21/08 |
| fase-12-errores.md           | Errores, logging y healthchecks               | Cerrada 22/08 |
| fase-13-reproducibilidad.md  | Reproducibilidad                              | Cerrada 22/08 |
| fase-14-experimentos.md      | Experimentos                                  | Cerrada 22/08 |
| fase-15-informe.md           | Informe y defensa                             | En curso      |
| fase-16-demostracion-ui.md   | La demostración de la carrera en el tablero   | Cerrada 23/08 |

La Fase 1 no tiene bitácora propia: fue el prototipo desechable de cola y trabajadores que vive en demo/ con su propio README, y su resultado (reparto 5/5/5 con tres trabajadores iguales, sin duplicados) quedó registrado ahí. La Fase 2 aparece como «diseño listo» y no «cerrada» a propósito: la prueba de equipo que el plan exige para darla por terminada —ambos integrantes respondiendo el cuestionario de docs/diseno.md §6 sin mirar el documento— es la única condición de cierre que quedó pendiente.